Starling

Animated renderings of data structures for teaching

Manual for v0.2.0

Contents

1. Introduction

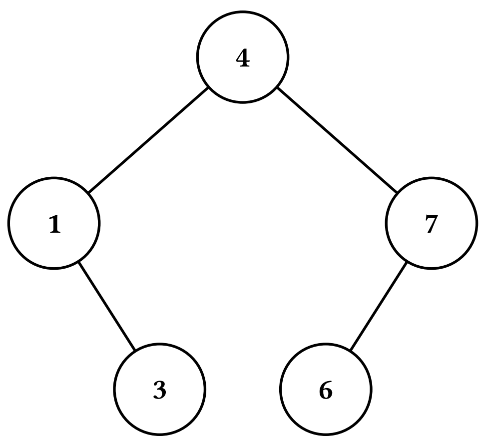
Starling renders animated data structures in Typst, built on top of cetz, typsy, and (optionally) touying. The package is designed for a programming course: it ships a binary search tree today and is structured to grow to heaps, hash tables, and graphs without rewriting the animation kernel.

Every animation in starling is built from the same primitive — an ordered array of `Frame` records. Each `Frame` carries a rendered cetz canvas, an optional textual caption, and free-form step metadata. The package supplies a few helpers to collapse the array into a final image, a vertical stack, or an array of subslide figures, but you are free to ignore them and lay frames out however your document needs.

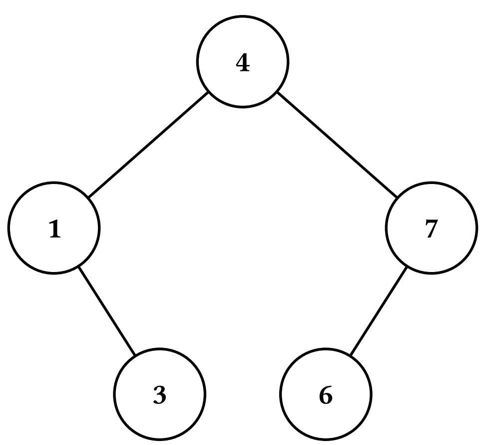
1.1. Quick start

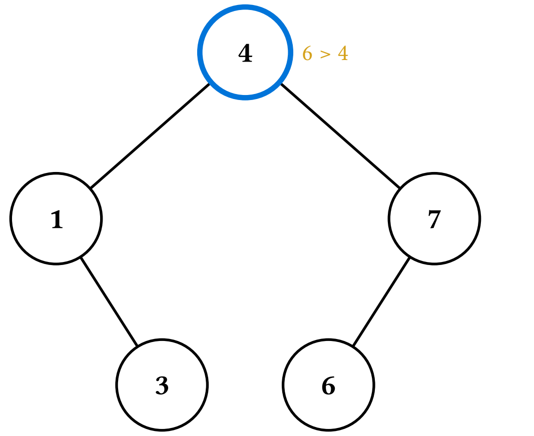
```
#import "@preview/starling:0.2.0" as starling
#import starling: BST

#let t = (BST.new)(value: 4, label: auto, left: none, right: none)
#let t = (t.insert)(1)
#let t = (t.insert)(7)
#let t = (t.insert)(3)
#let t = (t.insert)(6)
```

A static render of the tree, taking the final frame from `(t.display)()`:

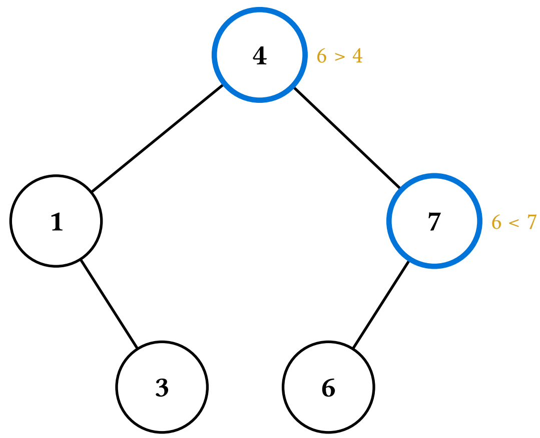


A full search animation, stacked vertically with per-step captions:

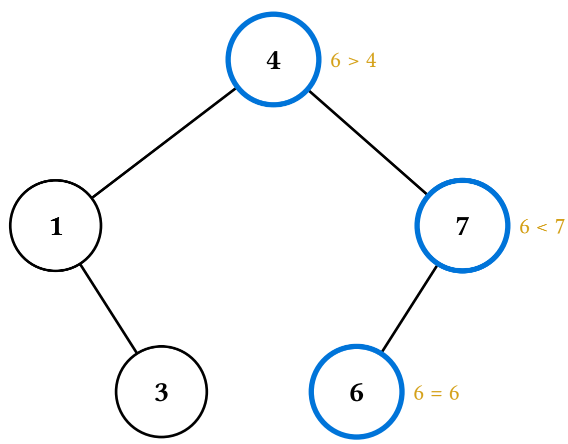




6 > 4



6 < 7



6 = 6

2. Architecture

Starling is layered into three source files, each with a single responsibility:

File	Role
`src/tree-anim.typ`	The animation kernel — defines `Snapshot`, `Frame`, `TreeRenderer`, `draw-tree`, `Op`, and `apply-ops`. Knows nothing about binary search trees specifically.

File	Role
`src/bst.typ`	The BST class — implements pure operations (`insert`, `delete`, `rotate`) and the `*-display` methods that build animations using the kernel.
`src/lib.typ`	The public surface — re-exports everything users need, plus the render helpers (`last`, `stacked`, `figures`, `canvases-only`).

This split lets us add a new data structure (e.g. a heap) by writing just a new file at the `bst.typ` layer; the kernel does not need to change. Conversely, a power user who wants to assemble custom animations can talk to the kernel directly without touching the BST class at all.

2.1. Snapshots, frames, and the renderer

Four types do most of the work. Their roles are intentionally distinct:

Type	Role
`Snapshot`	A *sparse style overlay* for one moment in time — which nodes are filled what colour, which edges are dashed or hidden, which notes are attached to which nodes. Built up by chaining `style-node` / `style-edge` / `note-node` calls. Pure data; no theme awareness, no content.
`TreeRenderer`	A tree plus an ordered list of snapshots plus optional caption / step-metadata for each snapshot. The thing you build up while describing an animation.
`Frame`	The *output* record: `(render, caption, step, alt)`. `render` is a builder function `(bst-theme, render-theme) -> content` — not pre-baked content — so callers can resolve theme state at layout time and reuse the same animation across documents with different themes. `TreeRenderer.render` produces an array of these, one per snapshot. This is what `*-display` methods return.
Theme dict	Either a `render-theme` (structural defaults like node fill, edge stroke, note colour) or a `bst-theme` (semantic strokes/fills like `search-stroke`, `success-fill`). Frames are theme-agnostic until a render helper feeds resolved themes into each frame's `render` builder.

The data flow, end to end:

`Snapshot` array $\longrightarrow$ `TreeRenderer.render`
$\longrightarrow$ `Array(Frame)` *(builders, theme-agnostic)*
$\longrightarrow$ helper resolves theme state once
$\longrightarrow$ helper calls each `(f.render)(bt, rt)`
$\longrightarrow$ document content

The shape matters: snapshots and frames are both pure data until the final step. That makes them inspectable, slicable, and composable — you can pull a frame out, look at its `step` metadata, swap its caption, or thread it into a custom layout without rendering. The theming-aware materialisation is concentrated at the helper boundary.

2.2. Path identity

Every node is identified by its position in the tree, encoded as a string of `"L"` and `"R"` characters from the root. The root is `""`, the right child of the root is `"R"`, the left child of `"R"` is `"RL"`, and so on. Edges are identified by the path of their *child* node — each child has exactly one parent, so this is unambiguous.

Path identity is the linchpin that keeps node and edge styling independent of the tree's value or layout. It also means a node-highlighting animation built for one tree can be reused on a tree of the same shape with different values just by passing a different `tree` to `make-renderer`.

The only places that interpret path characters are `PathId` (the refined string type), `_build-cetz-tree` (which walks `.left` and `.right`), and the BST's own `by-value` / `path-to` / `resolve` methods. Everywhere else treats paths as opaque keys, which leaves room to generalise — n-ary trees would use slash-separated child indices like `"0/1/2"` or arrays of ints, and only those four places need to change.

2.3. Sticky animation accumulation

`make-renderer(tree, sticky: true)` is the common case: each new frame pushed via `r.push-frame()` starts from the previous frame's snapshot. That means a sequence like

```
#let r = starling.make-renderer(t, sticky: true)
#let r = (r.push-with-node)("",  stroke: blue + 2pt)   // frame 1
#let r = (r.push-with-node)("L", stroke: blue + 2pt)   // frame 2
#let r = (r.push-with-node)("LR", stroke: blue + 2pt)  // frame 3
```

produces three frames where each highlights one more node than the last — frame 3 has all three highlights, not just `LR`. This matches the natural mental model for teaching ("we visited these nodes in order"). Set `sticky: false` if you want each frame to start fresh.

Captions and `step` metadata do *not* sticky-accumulate; each frame's caption is whatever was last set on that frame, with no carry-over.

2.4. The cetz integration

The animation kernel renders each snapshot via `draw-tree`, which emits cetz draw commands using `cetz.tree.tree(...)` for layout. The key choice here is *not* to wrap `draw-tree`'s output in `cetz.canvas` inside the kernel — that wrap happens one layer up, in `_render-canvas`. The two functions are split so a power user can write their own `cetz.canvas` block, call `draw-tree` inside it, and add their own cetz annotations alongside the tree (see Section 7).

Anchors for individual nodes are exposed by `path-anchor`, which translates an L/R path string into the cetz anchor name produced by `draw-tree`. The fact that this needs a translation function at all is a quirk worth flagging — see Section 3.7.

3. Design decisions

This section explains the major calls that shape the API. None of them are obvious from reading the code, but each was a conscious choice and the reasoning is load-bearing for anyone considering a refactor.

3.1. Why frames, not a wrap callback

The original (v0.1) API let users configure `starling.configure(wrap: alternatives)` to bake touying's `alternatives` into the BST class via closure capture. Each `*-display` method internally called `wrap(..figs)` on the array of figures it generated.

That approach was rigid in two ways. First, the rendering decision was made *inside* the animation method, so the same animation could not appear as a static figure in handouts and as a subslide sequence in slides — you'd configure differently per context. Second, and worse, the animation was always one opaque blob: there was no way to weave the caption "6 > 4" alongside other slide content while keeping the visual animation synchronised, because everything lived inside a single `alternatives` mark.

The current API solves both: `*-display` returns the raw frame array, and the helpers (`last`, `stacked`, `figures`) collapse it however the caller wants. Wanting to lay captions in a sidebar while the animation plays? Pull them out yourself:

```
#let frames = (t.search-display)(6)
#grid(columns: 2,
  alternatives(..starling.figures(frames, caption: false)),
  alternatives(..frames.map(f => f.caption)),
)
```

A historical note worth preserving: closure-captured `wrap` was itself chosen over a state-based approach (`std.state`, typsy's `safe-state`) because touying's `alternatives` is a layout-time "mark" that touying rejects inside `context { ... }` blocks. With frames as plain data, that constraint no longer applies to the animation API — and state did become viable later for the theming layer (see *State for ergonomics, per-call for perf* below).

3.2. Two theme layers, one per concern

Theming is split into two dicts that live in different files:

- **Render theme** (`default-render-theme`, in `tree-anim.typ`) holds structural defaults — node fill, node stroke, node text fill, edge stroke, note fill. Anything that a future data structure (heap, trie, B-tree) would also need.
- **BST theme** (`default-bst-theme`, in `bst.typ`) holds semantic strokes and fills tied to BST operations — `search-stroke`, `pivot-stroke`, `success-stroke`, `settled-stroke`, `success-fill`, `danger-stroke`, `reset-stroke`, `traversal-palette`.

The split lets each layer own its own concerns. When a new data structure lands, it brings its own semantic theme dict (`default-heap-theme`, say) and reuses the render-theme as-is. If the structural defaults were tangled into the BST theme, every new data structure would either duplicate them or grow a coupling to BST.

The render theme also has a clear "lowest in the chain" role: `draw-tree` falls back to it when neither a snapshot override nor a `make-renderer(default-node-style:, default-edge-style:)` argument specifies a property. So the merge order, lowest precedence first, is: `default-render-theme` ⟶ user render-theme override ⟶ renderer defaults ⟶ per-snapshot per-path overrides. Each layer adds more specificity.

Strokes throughout the BST theme are full stroke dictionaries (`(paint:, thickness:, dash:)`), not bare colours. That lets users change any aspect of a stroke (dash pattern, cap, width) without us adding more theme keys for each possibility.

3.3. Frames carry builders, not pre-rendered content

`Frame.render` is a function `(bst-theme, render-theme) -> content`, not a piece of pre-baked content. This is so render helpers (`last`, `stacked`, `figures`) can resolve theme state *once per call* and feed those resolved themes into every frame's builder, rather than each frame independently resolving theme inside its own context block.

The frame-as-builder shape also has a non-perf benefit: an `Array(Frame)` is now a portable, theme-agnostic description of an animation. The same frames can render against different themes in different contexts without re-running the `*-display` method. That's a cleaner separation than the older `(canvas, caption, step)` shape, where `canvas` baked in whatever theme was active at construction time.

A typsy quirk worth noting: typsy auto-injects `self` into any function-typed field on a class. To keep `(frame.render)(bt, rt)` from getting a spurious third argument, the actual closure is stored as `_builder: (fn: ...)` — a singleton dict around the function — and exposed through a `render` method that dereferences it. Users only see `(frame.render)(bt, rt)`; the dict wrap is private.

3.4. State for ergonomics, per-call for perf

Theme overrides come in two flavours:

Form	**Behaviour**
`set-bst-theme((..))` / `set-render-theme((..))`	State-based. One declaration at the top of the document propagates through every subsequent `*-display` call. Ergonomic and intent- matching, but participates in Typst's state-convergence machinery — see Section 5.3.
`(t.search-display)(v, theme: (..))`	Per-call. Overrides the theme for one call; ignores state for that call. Verbose if you have many calls, but skips the state-cost path entirely. Run at the no-state baseline.

Both paths exist because there's no single answer. State is right for a document that uses one theme throughout — write `set-bst-theme(my-palette)` once, forget about it. Per-call is right when compile speed dominates and the user is happy threading a `let palette = (..)` value through their calls. The library does not pick a winner.

The cost being structural to Typst — not something starling can optimise away — meant choosing whether to expose state at all was a real call. Skipping state would have removed the perf footgun but also removed the ergonomic win. Exposing both, with the cost documented up-front, lets users decide.

3.5. Two-channel captions

For `search-display` and `insert-display`, the comparison string ("6 > 4") appears in *two* places:

- **Inline**, as a small note drawn beside the highlighted node in the cetz canvas. This labels which comparison happened at which node spatially.

- **Caption**, as a textual track on the `Frame` record. This is what `stacked` and `figures` render below the canvas, and what callers pulling captions out for custom layouts read.

The redundancy is deliberate. The inline note is part of the visual: when looking at a single frame, you should be able to see at a glance what the algorithm just did. The caption is a parallel *textual* track for layouts that want narration apart from the animation.

This is why `last(frames)` defaults to `caption: false` — the inline note already labels the node, so adding a caption block below the static figure would be redundant. `stacked` and `figures` default to `caption: true` because animated rendering invites the textual narrative.

3.6. The six-frame rotation

`rotate-display` walks through rotation as six discrete frames: `init`, `pivots`, `break`, `restructure`, `connect`, `settle`. The sequence is calibrated for teaching: each frame answers exactly one question.

`step.kind`	**Question answered**
`init`	What's the starting tree?
`pivots`	Which two nodes are rotating?
`break`	Which edges are about to disappear?
`restructure`	What does the new shape look like, edges aside?
`connect`	Where do the new edges go?
`settle`	What does the final, clean tree look like?

An earlier draft had a "dashed red" intermediate between `pivots` and `break` (edges going dashed before disappearing). That was dropped because the orange pivot highlight already cues the eye to those edges — the extra frame added length without information.

The `restructure` frame is the load-bearing one: it shows the new tree shape with the rotated edges still hidden, so the student can see the new positions form *before* the edges connect. Without it, the rotation would collapse into one cut from "broken" to "rotated + green edges", which is too much information at once.

3.7. Path-based anchors and the cetz tree quirk

`cetz.tree.tree` numbers its nodes internally as `0`, `0-0`, `0-1`, `0-0-1`, … — sibling indices joined by hyphens, with the root being `0`. Starling configures cetz-tree's `group-name-prefix` to `"node-"`, so the resulting anchor names look like `node-0-0-1`. The whole tree also sits inside an outer named group (`"tree"` by default), so the fully qualified anchor for path `"LR"` is `tree.node-0-0-1`.

Two things made me reach for `path-anchor` rather than letting users type that out:

1. The user thinks in `"LR"`, not `"0-0-1"`. The translation is mechanical (`L → 0`, `R → 1`, prepend the root `0`) but it's error-prone to do by hand.
2. The naming scheme is a cetz-tree implementation detail. Wrapping it in `path-anchor(path, tree-name:, prefix:)` lets us swap to a different naming scheme later (e.g. if we drop cetz-tree for a custom layout) without churn at every annotation call site.

Phantom siblings (used to off-centre lone children — see `_build-cetz-tree`) also get cetz-tree-internal anchor names like `node-0-0-0`, but they are hidden at draw time and you generally should not annotate them.

4. BST animations tour

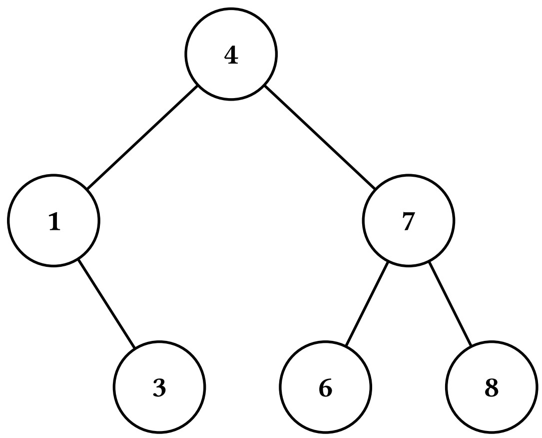
The BST class provides five `*-display` methods. Each returns `Array(Frame)`; the examples below pass the result to `starling.stacked` to render every frame vertically with its caption.

We'll use the same sample tree throughout this section:

```
#let t = (BST.new)(value: 4, label: auto, left: none, right: none)
#let t = (t.insert-many)(1, 7, 3, 6, 8)
```

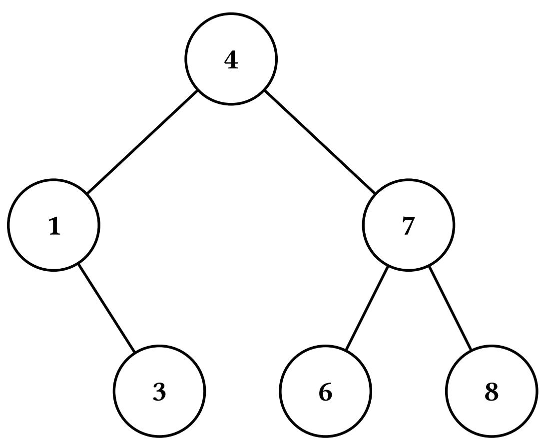
4.1. Static display

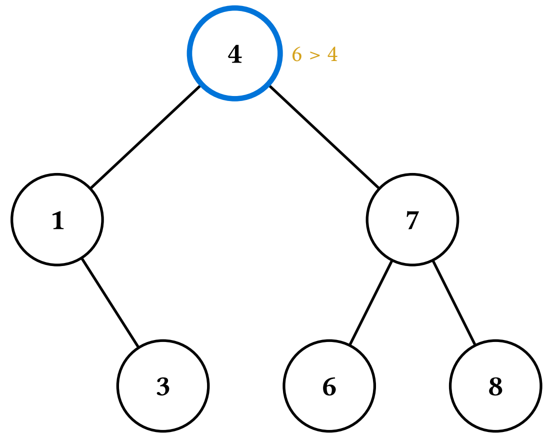
`(t.display)()` returns a one-element frame array — the unmodified tree, with no step metadata.



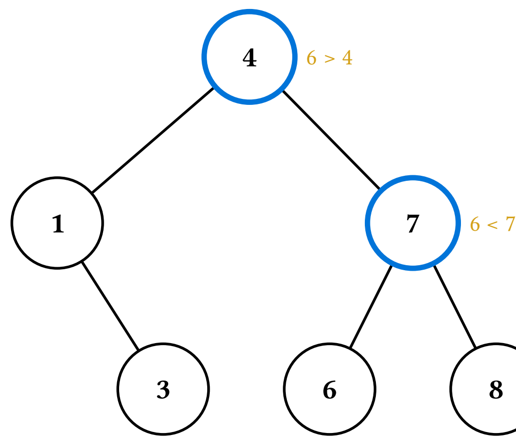
4.2. Search

`(t.search-display)(v)` walks the search path, highlighting each node visited and labelling it with the comparison made. `step.kind` is `"init"` for the initial frame and `"compare"` for each step; `step.found` records whether the value was reached.

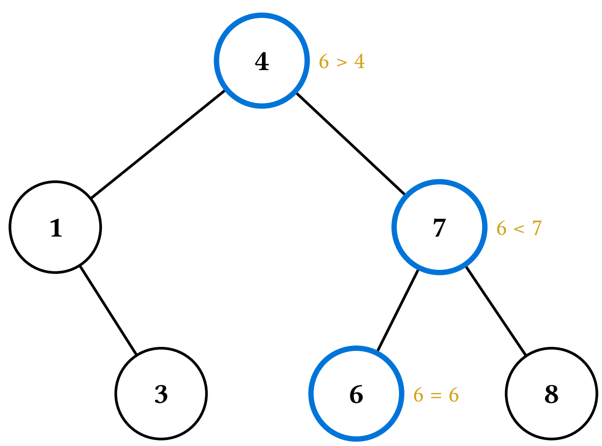




6 > 4



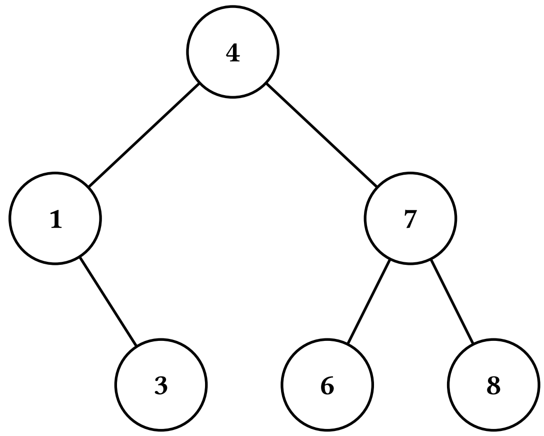
6 < 7

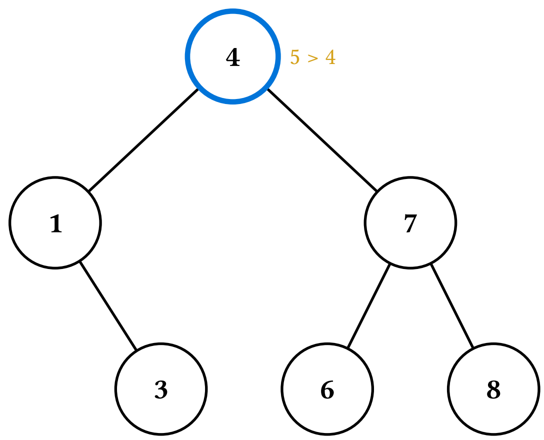


6 = 6

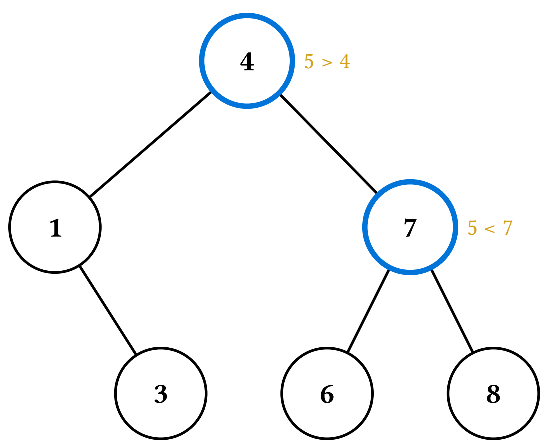
4.3. Insert

`(t.insert-display)(v)` walks the search path as for `search-display`, then transitions to the after-tree with the new node highlighted green. `step.kind` is `"init"`, `"compare"` (per search step), and finally `"inserted"`.

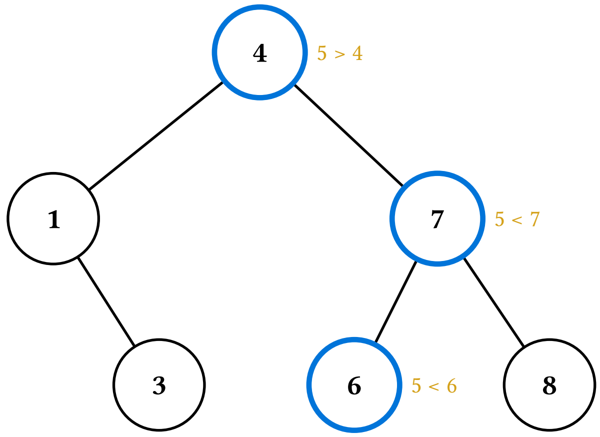




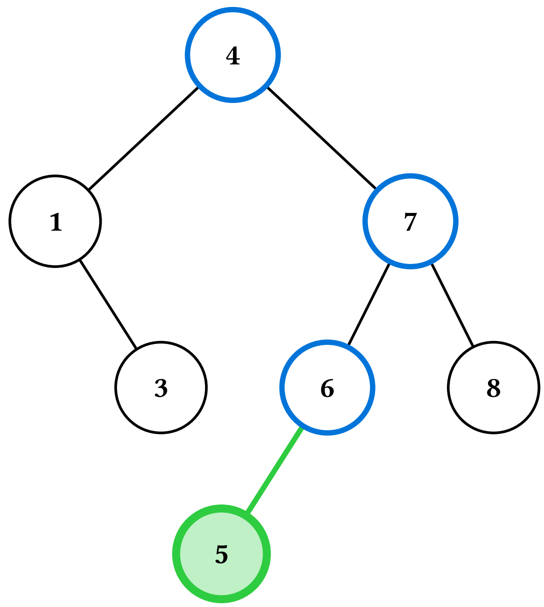
5 > 4



5 < 7



5 < 6

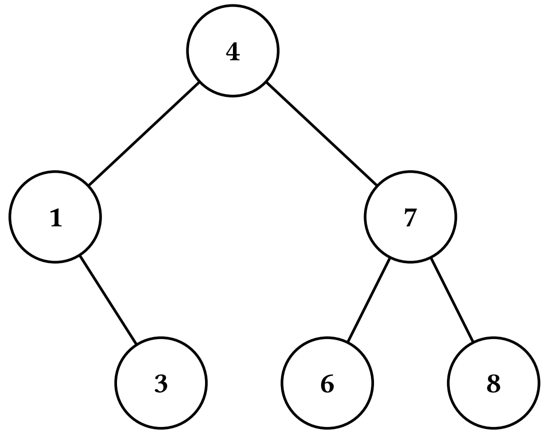


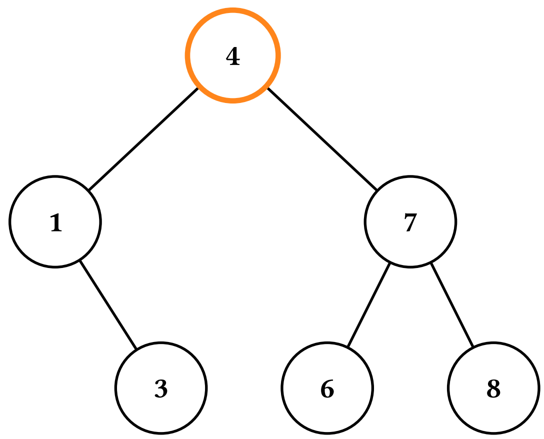
Inserted 5

4.4. Delete

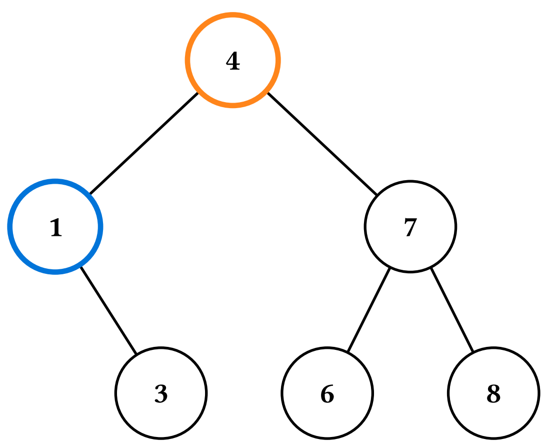
`(t.delete-display)(v)` has three internal cases — leaf, one-child, and two-children — and dispatches on the target's children. `step.kind` ranges over `"init"`, `"highlight"`, `"break"`, `"descend"`, `"transfer"`, and `"settle"` depending on case.

The two-children deletion is the most complex: it highlights the target, descends into the left subtree to find the in-order predecessor, annotates the value transfer, then jumps to the after-tree with the new root of the affected subtree highlighted green.

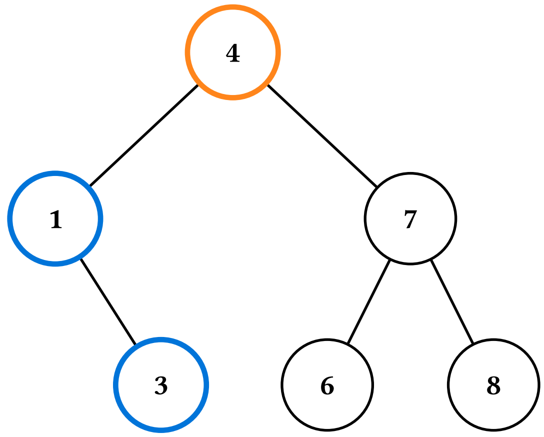




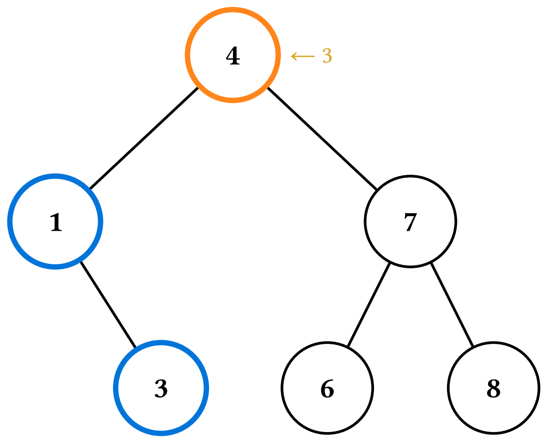
Delete 4



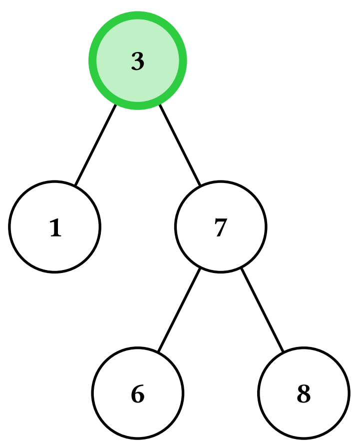
Find predecessor



Find predecessor



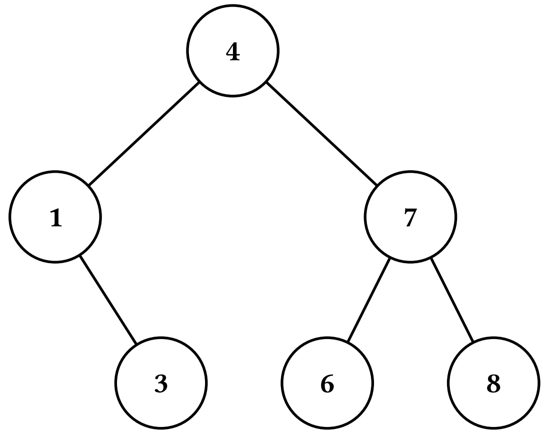
Transfer 3

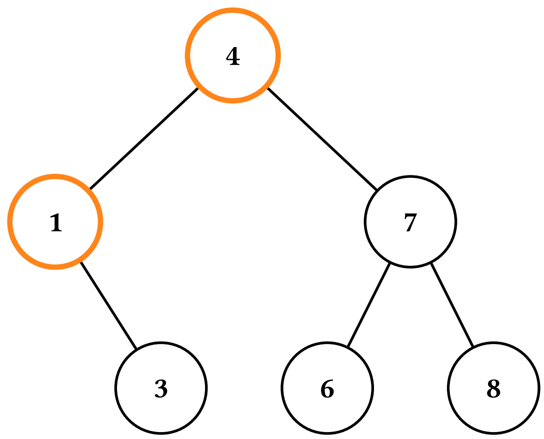


Done

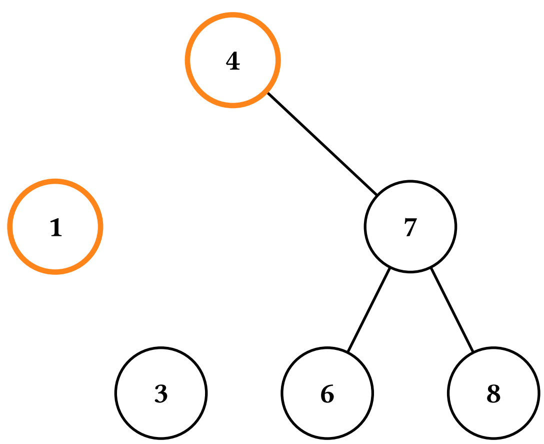
4.5. Rotate

`(t.rotate-display)(c)` rotates around the direct child whose value is `c`. The six-frame sequence is described in Section 3.6.

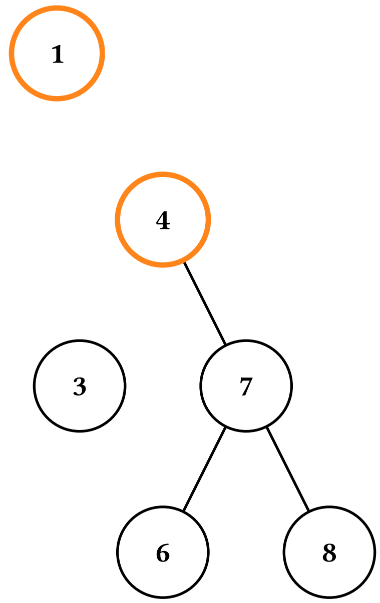




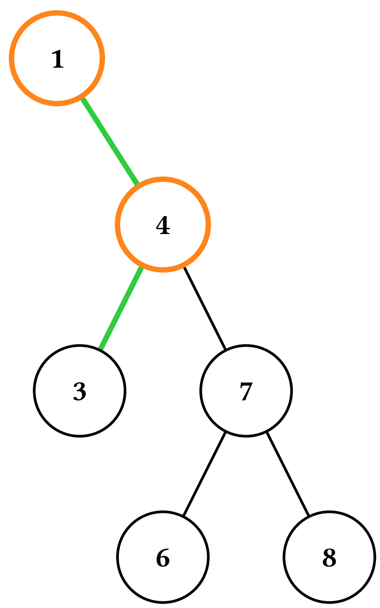
Rotate around 1



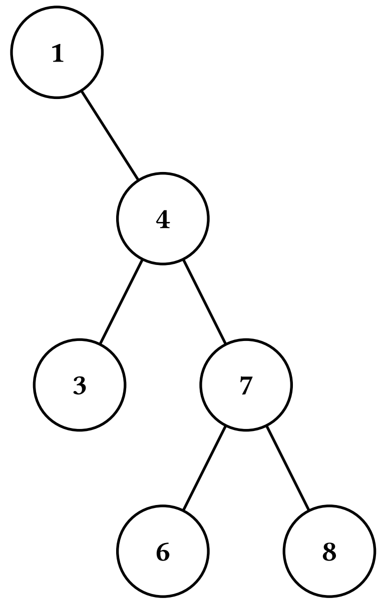
Break edges



Restructure tree



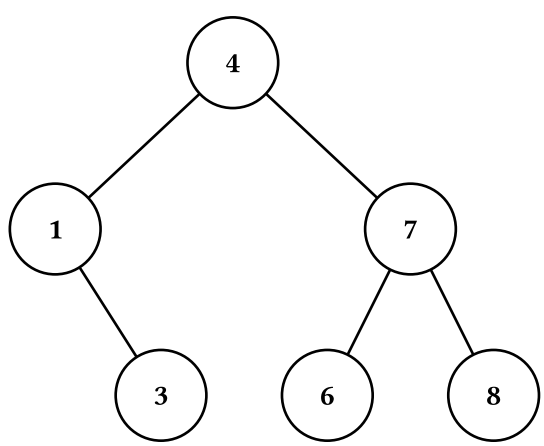
Reconnect edges

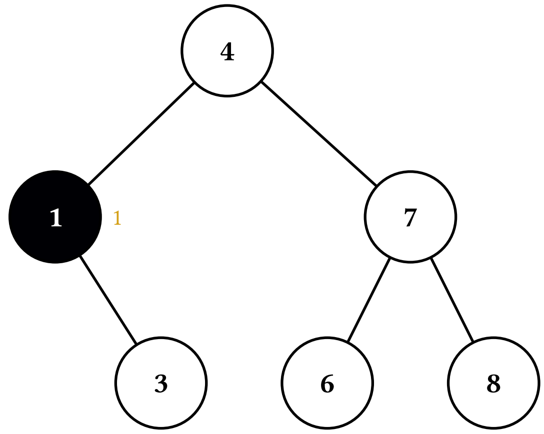


4.6. Traversals

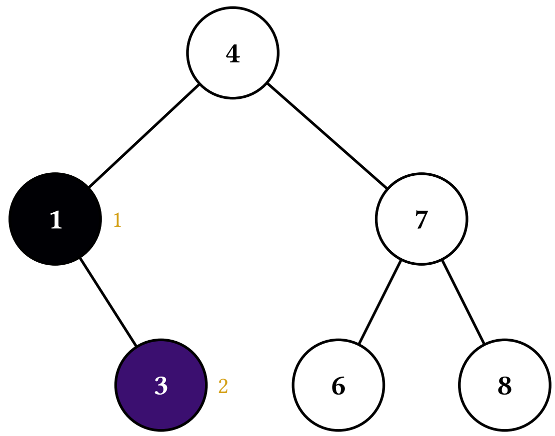
`(t.in-order-display)()`, `(t.pre-order-display)()`, `(t.post-order-display)()`, and `(t.level-order-display)()` animate the four standard traversals. Each returns one frame per visit (plus the initial frame): the visited node is filled with the next color in a perceptually-uniform palette (magma by default) and tagged with a numbered badge marking its visit order, and the caption accumulates the running output sequence. `step.kind` is `"init"` for the initial frame and `"visit"` thereafter, with `step.value` and `step.index` (1-indexed) recording each visit. Switch palettes via the theme system (see Theming) — either doc-wide with `set-bst-theme((traversal-palette: color.map.viridis))` or per-call with `(t.in-order-display)(theme: (traversal-palette: ...))`.

Pure-data variants `(t.in-order)()`, `(t.pre-order)()`, `(t.post-order)()`, and `(t.level-order)()` return the visit sequence as an array of path strings without producing any animation, in case you want to drive a custom layout.

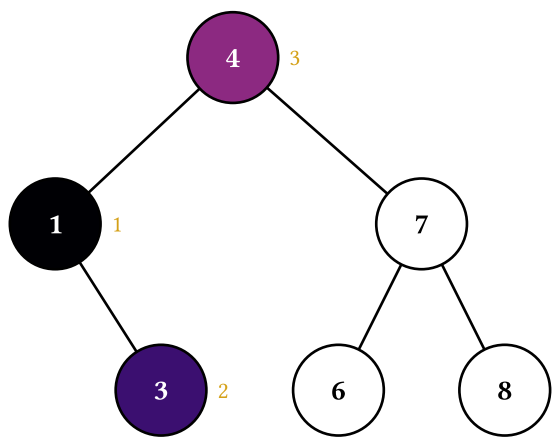




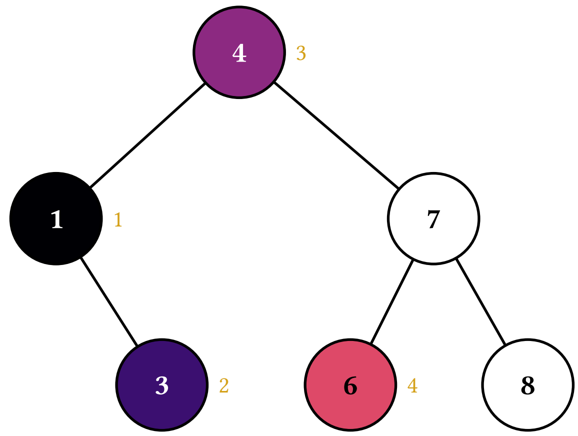
Output: [1]



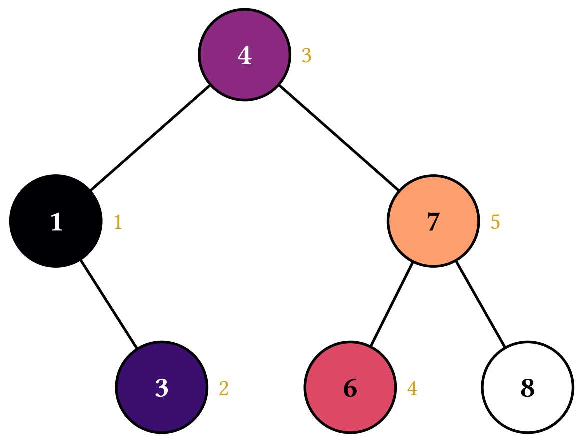
Output: [1, 3]



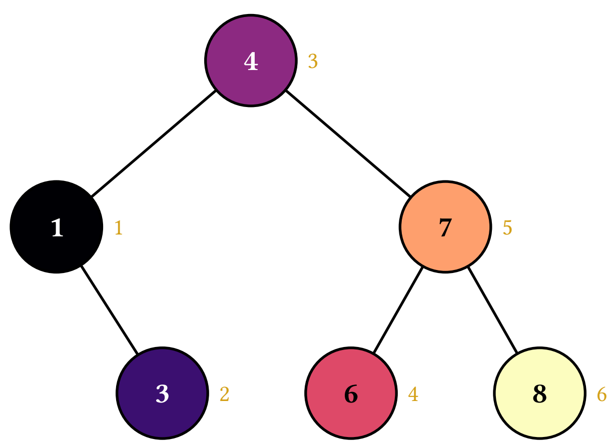
Output: [1, 3, 4]



Output: `[1, 3, 4, 6]`



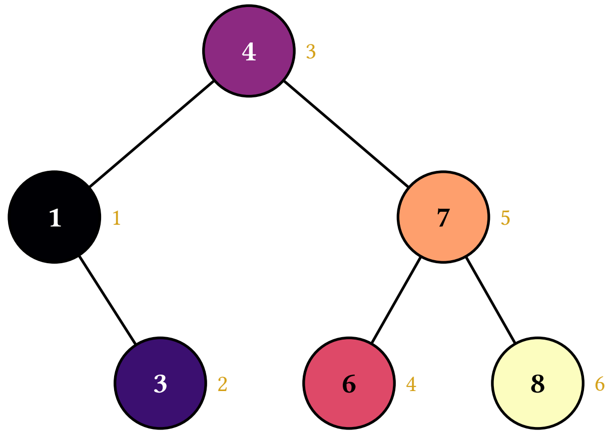
Output: `[1, 3, 4, 6, 7]`



Output: `[1, 3, 4, 6, 7, 8]`

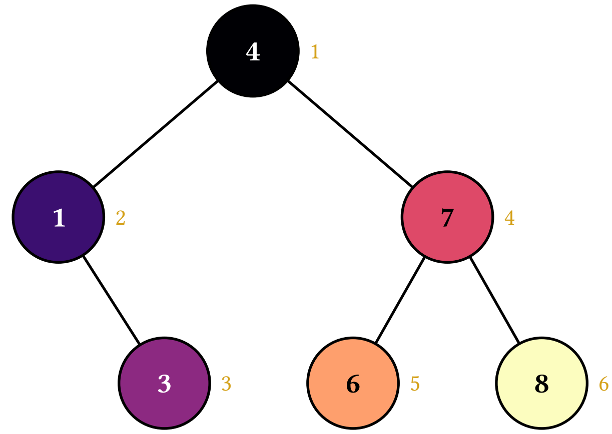
The final frame of each traversal carries the algorithm's full color signature. Laying all four out side-by-side gives a fast visual comparison — especially useful as a recall cue in review material where students already understand the algorithms and the gradient pattern reads as "oh right, post-order goes leaves-cool to root-warm":

In-order



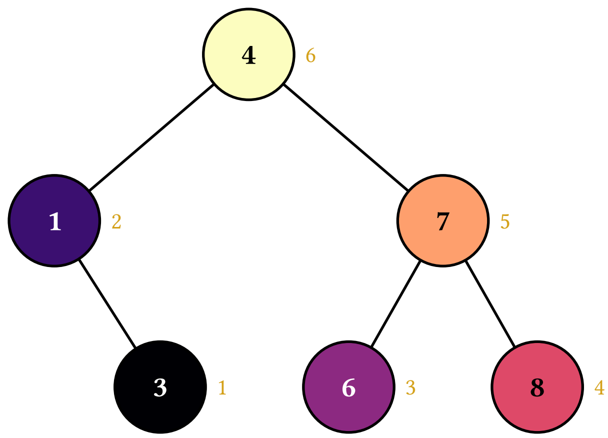
Output: `[1, 3, 4, 6, 7, 8]`

Pre-order



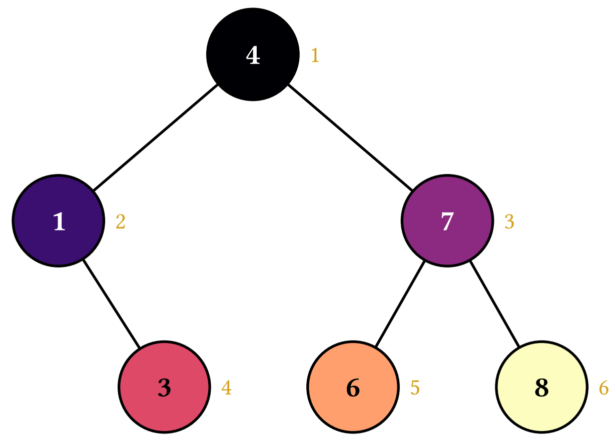
Output: `[4, 1, 3, 7, 6, 8]`

Post-order



Output: `[3, 1, 6, 8, 7, 4]`

Level-order



Output: `[4, 1, 7, 3, 6, 8]`

5. Theming

Starling exposes two theme layers so document authors can match starling's palette to their own document style:

- **Render theme** (`default-render-theme`) — the structural defaults the renderer falls back on for the unstyled tree: `node-fill`, `node-stroke`, `node-text-fill`, `edge-stroke`, `note-fill`.
- **BST theme** (`default-bst-theme`) — the semantic colors and strokes the BST `*-display` methods use to communicate operation state: `search-stroke`, `pivot-stroke`, `success-stroke`, `settled-stroke`, `success-fill`, `danger-stroke`, `reset-stroke`, `traversal-palette`. Strokes are full stroke dicts (`(paint:, thickness:, dash:)`) so any aspect — color, width, dash — is overridable without adding more keys.

5.1. Setting a theme for the whole document

Call `set-bst-theme` and/or `set-render-theme` once near the top of your document. The override is state-based and scoped by Typst's normal layout flow, so it propagates to every subsequent `*-display` call.

```
#import "@preview/starling:0.1.0": BST, set-bst-theme, set-render-theme

#set-bst-theme((
  search-stroke: (paint: teal, thickness: 2.5pt),
  success-stroke: (paint: olive, thickness: 2.5pt),
  settled-stroke: (paint: olive, thickness: 3.5pt),
  success-fill: olive.lighten(75%),
))
#set-render-theme((node-fill: yellow.lighten(85%)))
```

Pass a partial dictionary — only the keys you list are changed. Unknown keys panic so typos surface immediately rather than silently falling back to defaults.

5.2. Per-call overrides

For one-off variations, every `*-display` method accepts `theme:` and `render-theme:` arguments. A partial dict is merged into the defaults and used in place of the state value for that call only.

```
(t.search-display)(6, theme: (search-stroke: (paint: olive, thickness: 2pt)))
```

5.3. Performance: state-based theming costs a layout pass

Typst's state machinery is what makes `set-bst-theme` / `set-render-theme` work: a `state.update` later in the document can affect renders earlier in document order, so Typst evaluates the document, propagates state, and re-evaluates anything that observed it. In practice that means **using either state setter roughly doubles the compile time of the state-observed portion of the document** — a small fixed cost on top of each tree, plus a full extra layout pass through everything that placed a starling render helper or rendered a frame's `render` builder against that state.

Concretely, a 50-tree synthetic benchmark goes from 1.85s to 3.1s when `set-bst-theme` is added. The cost scales with document size, not with the number of state reads (one read or a thousand is the same), because the price is "Typst does a second layout pass", not "the read is slow".

This is a structural property of Typst's state model, not something starling can work around without giving up state. If compile speed matters more than the ergonomic win, hoist your theme dict into a plain `let` and pass it to each `*-display` call via the per-call `theme:` argument — that path skips state entirely and runs at the no-state baseline:

```
#let palette = (
  search-stroke: (paint: teal, thickness: 2.5pt),
  success-stroke: (paint: olive, thickness: 2.5pt),
  // ...
)

#starling.stacked((t.search-display)(6, theme: palette))
#starling.stacked((t.insert-display)(5, theme: palette))
// ...
```

This is the usual perf/ergonomics tradeoff: `set-bst-theme` is one declaration at the top of the document and "just works"; per-call `theme:` is more typing but avoids the extra pass.

6. Touying composition examples

Starling is layout-agnostic. In a touying deck, the simplest use splats `figures` into `alternatives`:

```
#import "@preview/touying:0.7.3": *
#import "@preview/starling:0.2.0" as starling
#import starling: BST

== Searching
#alternatives(..starling.figures((t.search-display)(6)))
```

Each frame becomes a subslide; touying handles the rest.

6.1. Captions in a sidebar

When the visual animation should sit alongside synchronised text elsewhere on the slide, pull the captions out separately:

```
== Searching for 6
#let frames = (t.search-display)(6)
#grid(columns: 2, column-gutter: 2em,
  alternatives(..starling.figures(frames, caption: false)),
  alternatives(..frames.map(f => f.caption)),
)
```

Both `alternatives` calls produce the same number of subslides, so they step in lockstep.

6.2. Driving layout from step metadata

`frame.step` is free-form per-method metadata. For example, a slide that wants to colour-code its narration by step kind:

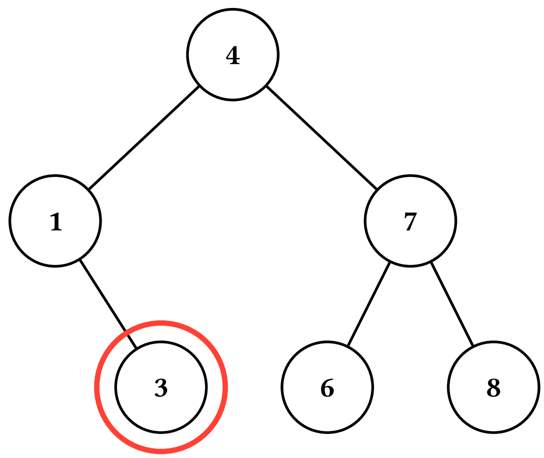
```
#let kind-colors = (compare: blue, inserted: green)

#let captioned-frames = frames.map(f => figure(context {
  let bt = starling._bst-theme-state.get()
  let rt = starling._render-theme-state.get()
  let color = kind-colors.at(f.step.kind, default: black)
  stack(dir: ttb, spacing: 0.5em,
    (f.render)(bt, rt), text(fill: color, f.caption))
}))

#alternatives(..captioned-frames)
```

`frame.render` is a builder `(bt, rt) => content` rather than pre-baked content, so themes resolve at layout time. The `context` block above reads the active themes and feeds them in; if you don't need state-driven theming, you can call `(f.render)(starling.default-bst-theme, starling.default-render-theme)` without the wrapper.

7. Composing with cetz annotations

For callouts or overlays that need to track specific nodes, drop down to the cetz layer. `draw-tree(tree, snapshot, ...)` emits the tree's cetz drawables without wrapping them in `cetz.canvas`, so you can compose them with your own draw commands in a shared canvas. `path-anchor(path)` translates an L/R path string to the cetz anchor name of the corresponding node.

```
#import "@preview/cetz:0.5.2"

#cetz.canvas({
  starling.draw-tree(t, starling.blank-snapshot())
  import cetz.draw: *
  circle(starling.path-anchor("LR"), radius: 0.85, stroke: red + 2pt)
})
```



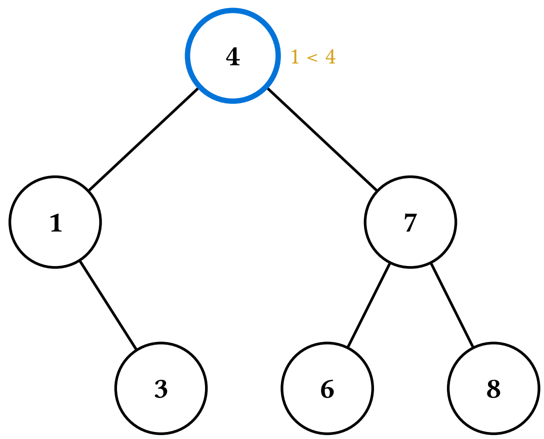
The same trick combines with frame canvases: each frame's `canvas` is itself a `cetz.canvas` block, so for static "annotate the final frame" use you can either render once via `draw-tree` (as above) or extract the cetz block from a frame and add to it externally.

8. The Op command stream

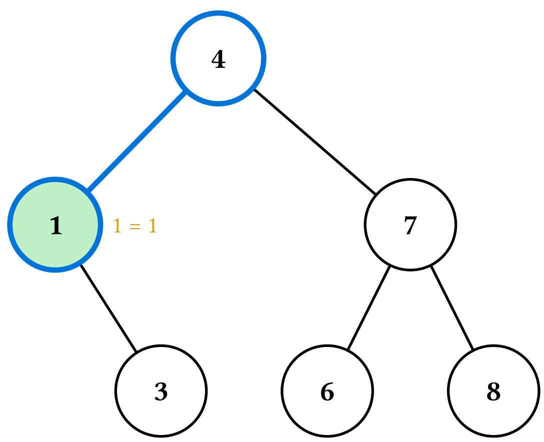
For animations that don't fit the BST's built-in methods — say, a custom hash-table probe sequence or a graph traversal — drop down to the `Op` command stream. Build a sequence of declarative ops and fold them into a renderer with `apply-ops`:

```
#let r = starling.make-renderer(t, sticky: true)
#let r = (r.with-caption)([start at root])
#let r = starling.apply-ops(r, (
  (starling.Op.Highlight.new)(path: "", color: blue),
  (starling.Op.Annotate.new)(path: "", text: [1 < 4]),
  (starling.Op.Commit.new)(
    alt: "Comparing the target 1 against the root 4.",
  ),
))
#let r = (r.with-caption)([descend left])
#let r = starling.apply-ops(r, (
```

```
  (starling.Op.ClearNotes.new)(),
  (starling.Op.Highlight.new)(path: "L", color: blue),
  (starling.Op.StyleEdge.new)(path: "L", style: (stroke: blue + 2pt)),
  (starling.Op.StyleNode.new)(path: "L", style: (fill: green.lighten(70%))),
  (starling.Op.Annotate.new)(path: "L", text: [1 = 1]),
  (starling.Op.Alt.new)(
    text: "Descended into the left subtree; 1 matches the left child.",
  ),
))
#starling.stacked((r.render)())
```



start at root



descend left

`Op.Commit` closes the current frame, attaches the supplied `alt` text to it, and opens a fresh blank frame. The `alt` argument is required so every frame the command stream produces carries accessible text. The trailing in-progress frame is kept implicitly — no final `Op.Commit` is needed — but use `Op.Alt(text)` (or `r.with-alt(...)` on the returned renderer) to give that last frame its alt text too.

`Op.Caption` does *not* exist; captions and step metadata are renderer-level, set via `r.with-caption(...)` / `r.with-step(...)` directly between Op batches. The split keeps each Op's responsibility narrow (modifying the current snapshot) and matches how the `*-display` methods are written internally.

9. API reference

The remainder of this manual is an auto-generated reference, produced by the `tidy` package from doc-comments in the source.

9.1. Render helpers

- canvases-only()
- figures()
- last()
- stacked()

9.1.1. canvases-only

Drop captions and step metadata, returning just the array of canvases. Useful when you want to lay out captions yourself — e.g. alongside other slide content in a custom touying layout.

Like `figures()`, this returns one piece of content per frame and therefore can't amortize theme state reads — each canvas resolves state independently. If you're calling this on hot paths under a state-set theme, prefer per-call `theme:` arguments on the `*-display` method.

9.1.1.1. Parameters

```
canvases-only(frames: array) -> array
```

9.1.1.1.1. frames `array`

The frame array returned by any `*-display` method.

9.1.2. figures

Build an array of `figure` content, one per frame, suitable for splatting into touying's `alternatives(..)` so each frame becomes its own subslide. Starling itself does not depend on touying — the result is just an array of standard Typst figures.

Unlike `last()` / `stacked()`, this helper can't collapse theme state reads to one — each figure is an independent piece of content laid out separately (touying splits them into subslides), so each figure's body resolves theme state in its own `context` block. In many-tree documents that rely on `set-bst-theme` / `set-render-theme`, prefer per-call `theme:` arguments on the `*-display` method to skip state altogether when you're using `figures`.

9.1.2.1. Parameters

```
figures(
  frames: array,
  caption: bool,
  alt: auto str,
  caption-spacing: length
) -> array
```

9.1.2.1.1. frames `array`

The frame array returned by any `*-display` method.

9.1.2.1.2. caption `bool`

Whether to stack each frame's caption below its canvas inside the figure body.

Default: `true`

9.1.2.1.3. alt `auto` or `str`

Alt-text override applied to every frame. `auto` uses each frame's `alt` field, falling back to the caption (if it's a plain string) and then to a generic `"Animation frame N"` placeholder.

Default: `auto`

9.1.2.1.4. caption-spacing `length`

Spacing between canvas and caption when `caption: true`.

Default: `0.5em`

9.1.3. last

Final frame as a single piece of content. Useful for static renderings in print or for "just show me the end state" usage.

The inline node annotations (drawn next to nodes in the cetz canvas) already label what the final step did, so the textual caption is suppressed by default. Pass `caption: true` to stack the caption below the canvas.

The returned content is wrapped in an alt-tagged figure for accessibility. The alt text comes from the frame's `alt` field; pass an explicit string to `alt` to override.

9.1.3.1. Parameters

```
last(
  frames: array,
  caption: bool,
  spacing: length,
  alt: auto str
) -> content
```

9.1.3.1.1. frames `array`

The frame array returned by any `*-display` method.

9.1.3.1.2. caption `bool`

Whether to render the step's caption below the canvas.

Default: `false`

9.1.3.1.3. spacing `length`

Vertical spacing between canvas and caption when `caption: true`.

Default: `0.5em`

9.1.3.1.4. alt `auto` or `str`

Alt-text override. `auto` uses the frame's `alt` field.

Default: `auto`

9.1.4. stacked

All frames stacked vertically as one block of content. Useful for handouts that want to show the full animation in a single figure. Captions are on by default since the stack is read as a sequence — the caption text annotates which step each tree corresponds to.

Each frame is wrapped individually in an alt-tagged figure so screen-reader users hear the per-step narration. Pass a string to `alt` to set the same override on every frame.

9.1.4.1. Parameters

```
stacked(
  frames: array,
  caption: bool,
  spacing: length,
  caption-spacing: length,
  alt: auto str
) -> content
```

9.1.4.1.1. frames `array`

The frame array returned by any `*-display` method.

9.1.4.1.2. caption `bool`

Whether to include each frame's caption below its canvas.

Default: `true`

9.1.4.1.3. spacing `length`

Vertical spacing between frames.

Default: `1em`

9.1.4.1.4. caption-spacing `length`

Spacing between each canvas and its caption.

Default: `0.5em`

9.1.4.1.5. alt `auto` or `str`

Alt-text override applied to every frame. `auto` uses each frame's `alt` field.

Default: `auto`

9.2. Animation kernel

- apply-ops()
- blank-snapshot()
- concat-frames()
- draw-tree()
- make-renderer()
- path-anchor()
- set-render-theme()

Variables

- PathId
- NodeStyle
- EdgeStyle
- default-render-theme
- RenderTheme
- Snapshot
- Frame
- TreeRenderer
- Op

9.2.1. apply-ops

Fold a sequence of `Op` values into a renderer, returning the updated renderer. The trailing in-progress frame is kept — no explicit `Op.Commit` is needed after the last batch of edits, but attach alt text to that trailing frame via `Op.Alt` (or `r.with-alt(...)` on the returned renderer) so it has accessible text like the committed frames do.

9.2.1.1. Parameters

```
apply-ops(
  renderer: TreeRenderer,
  ops: array
) -> TreeRenderer
```

9.2.1.1.1. renderer `TreeRenderer`

The renderer to apply ops to.

9.2.1.1.2. ops `array`

Sequence of `Op` values to fold left-to-right.

9.2.2. blank-snapshot

Build a fresh `Snapshot` with no node or edge style overrides. Useful when you want to render a tree once with `draw-tree` and no per-frame state.

9.2.2.1. Parameters

```
blank-snapshot() -> Snapshot
```

9.2.3. concat-frames

Stitch frame arrays from multiple renderers (or pre-rendered arrays) into one flat `Array(Frame)`. Use this when an animation spans a tree-shape change (rotation, deletion, insertion) and one renderer can't hold all the frames — one renderer per shape, joined at the boundary.

Accepts a mix of `TreeRenderer` instances and already-rendered arrays; the former are `render`-ed in place.

9.2.3.1. Parameters

```
concat-frames(..parts) -> array
```

9.2.3.1.1. ..parts

Variadic mix of renderers and frame arrays.

9.2.4. draw-tree

Emit the cetz draw commands for one styled snapshot of `tree`, *without* wrapping them in a `cetz.canvas`. Caller is responsible for the canvas wrap. Use this when you want to add your own cetz annotations alongside the tree — for example, callouts anchored at specific nodes.

The whole tree is wrapped in a named cetz group (`name`, default `"tree"`); each non-phantom node has an inner-name of `<node-prefix><cetz-tree-name>` where the cetz-tree name is `"0"` for the root, `"0-0"` for L, `"0-1"` for R, `"0-0-1"` for LR, and so on. Fully qualified anchor names therefore look like `"tree.node-0-0-1"` — use `path-anchor()` to compute them from an L/R path string.

9.2.4.1. Parameters

```
draw-tree(
  tree: dictionary,
  snapshot: Snapshot,
  default-node-style: dictionary,
  default-edge-style: dictionary,
  render-theme: dictionary,
  name: str,
  node-prefix: str
) -> content
```

9.2.4.1.1. tree `dictionary`

The tree to render — any value with `value`, `label`, `left`, and `right` fields (e.g. a `BST` instance). `label` of `auto` falls back to `str(value)` any other value (string, image, content) is rendered as the node's drawn label.

9.2.4.1.2. snapshot `Snapshot`

Style overlay for this snapshot. Use `blank-snapshot()` for an unstyled tree.

9.2.4.1.3. default-node-style `dictionary`

Default node-style overrides applied before per-node overrides.

Default: `(:)`

9.2.4.1.4. default-edge-style `dictionary`

Default edge-style overrides applied before per-edge overrides.

Default: `(:)`

9.2.4.1.5. render-theme `dictionary`

Structural defaults — the lowest layer of the style chain. Defaults to `default-render-theme` pass an already-merged dict (e.g. read from `set-render-theme`'s state inside a `context` block) to apply user overrides.

Default: `default-render-theme`

9.2.4.1.6. name `str`

Outer group name for the whole tree. Must match the `tree-name` argument to `path-anchor()` for anchors to resolve.

Default: `"tree"`

9.2.4.1.7. node-prefix `str`

Prefix attached to each node's cetz-tree internal name. Must match the `prefix` argument to `path-anchor()`.

Default: `"node-"`

9.2.5. make-renderer

Build a `TreeRenderer` seeded with one blank initial frame.

With `sticky: true` (the default), each new frame pushed via `r.push-frame()` starts from the previous frame's style snapshot — so highlights accumulate over the course of an animation. Set `sticky: false` to start each frame from a clean slate.

With `theme: auto` (the default), each rendered canvas reads the active render theme from state at layout time, so a `set-render-theme(..)` earlier in the document propagates. Pass an explicit dictionary to bake a theme in and skip the state lookup.

9.2.5.1. Parameters

```
make-renderer(
  tree: dictionary,
  default-node-style: dictionary,
  default-edge-style: dictionary,
  sticky: bool,
  theme: auto dictionary
) -> TreeRenderer
```

9.2.5.1.1. tree `dictionary`

The tree to render — any value with `value`, `label`, `left`, and `right` fields. See `draw-tree()` for how `label` is rendered.

9.2.5.1.2. default-node-style `dictionary`

Default node-style overrides applied before per-snapshot overrides.

Default: `(:)`

9.2.5.1.3. default-edge-style `dictionary`

Default edge-style overrides applied before per-snapshot overrides.

Default: `(:)`

9.2.5.1.4. sticky `bool`

When `true`, new frames inherit the previous frame's style. When `false`, each new frame starts blank.

Default: `true`

9.2.5.1.5. theme `auto` or `dictionary`

Render-theme override for this renderer. `auto` reads the active render-theme from state at layout time. A dict is merged into `default-render-theme` once and baked in.

Default: `auto`

9.2.6. path-anchor

Translates a starling path-id (L/R string rooted at `""`) to the fully-qualified cetz anchor name produced by `draw-tree()`. Path `""` maps to the root anchor; `"L"`/`"R"` map to first-level children; and so on.

The `tree-name` and `prefix` arguments must match the `name` and `node-prefix` passed to `draw-tree`.

9.2.6.1. Parameters

```
path-anchor(
  path: str,
  tree-name: str,
  prefix: str
) -> str
```

9.2.6.1.1. path `str`

L/R path string identifying a node (`""` = root, `"LR"` = root's left child's right child, etc.).

9.2.6.1.2. tree-name `str`

Outer-group name; must match `draw-tree`'s `name`.

Default: `"tree"`

9.2.6.1.3. prefix `str`

Per-node prefix; must match `draw-tree`'s `node-prefix`.

Default: `"node-"`

9.2.7. set-render-theme

Override one or more render-theme keys for the rest of the document (state-based, scoped by Typst's normal layout flow). Pass a partial dictionary — only the keys you list are changed; the rest stay at their current values. Unknown keys panic.

9.2.7.1. Parameters

```
set-render-theme(theme)
```

9.2.8. PathId

Typsy refinement: a string built only from `"L"` and `"R"` characters. The empty string is the root; `"L"` is the root's left child; `"LR"` is the root's left child's right child; and so on.

9.2.9. NodeStyle

Typsy refinement: a dictionary of node-style overrides. Recognised keys are `fill`, `stroke`, `text-fill`, `note`, `note-fill`, and `hide`.

9.2.10. EdgeStyle

Typsy refinement: a dictionary of edge-style overrides. Recognised keys are `stroke`, `note`, `note-fill`, `mark`, and `hide`.

9.2.11. default-render-theme

Default render theme. The structural defaults the renderer falls back on when neither a snapshot nor `default-node-style`/ `default-edge-style` specifies otherwise.

9.2.12. RenderTheme

Typsy refinement: a dictionary whose keys are a subset of the render-theme keys (`node-fill`, `node-stroke`, `node-text-fill`, `edge-stroke`, `note-fill`).

9.2.13. Snapshot

Typsy class representing one sparse style overlay — the per-node and per-edge style overrides for a single frame. Build one via `blank-snapshot()` and chain its `style-node` / `style-edge` / `note-node` / `note-edge` / `clear-notes` methods. Snapshots are rarely constructed directly — they're managed by `make-renderer()` and the BST's `*-display` methods.

9.2.14. Frame

Typsy class representing one frame in an animation. Fields and methods:

- `render(bst-theme, render-theme)` — method that, given resolved theme dicts, produces the cetz canvas for this frame. Theming-aware deferral lives here: by carrying a builder rather than pre-baked content, the lib.typ helpers (`last`, `stacked`, `figures`) can resolve theme state once per call instead of once per frame, which is meaningfully faster in many-tree documents.
- `caption` — optional textual track for the step (possibly `none`).
- `step` — optional free-form per-method metadata (possibly `none`).
- `alt` — optional accessible text describing the frame (possibly `none`). Carried verbatim into the `alt:` argument on the Typst `figure` wrapping the canvas. The first frame of each operation includes the full tree structure (via the BST `describe` method) so screen-reader users have a baseline; subsequent frames describe only the per-step change.

Direct rendering pattern for custom layouts:

```
context {
  let bt = _bst-theme-state.get()
  let rt = _render-theme-state.get()
  ... (frame.render)(bt, rt) ...
}
```

Or simply pass the frame array to one of the lib.typ helpers, which handle the state wrap for you.

The internal `_builder` field carries the actual rendering closure wrapped in a singleton dict so typsy's auto-self-injection doesn't fire on it; treat it as private and call `render` instead.

9.2.15. TreeRenderer

Typsy class accumulating an animation. Holds the tree, parallel arrays of snapshots / captions / steps / alts (one entry per frame), and default node/edge styles. Methods include `push-frame()`, `patch(fn)`, `push-with-node(path, ..style)`, `push-with-edge(path, ..style)`, `push-note-node(path, txt)`, `push-note-edge(path, txt)`, `with-caption(c)`, `with-step(s)`, `with-alt(a)`, and `render()`. Build one via `make-renderer()`.

9.2.16. Op

Typsy enumeration describing a declarative command stream for building animations. Variants:

- `Op.Highlight(path, color)` — set a node's stroke to `color + 2pt`.
- `Op.Annotate(path, text)` — attach an inline note to a node (drawn in-canvas).
- `Op.StyleNode(path, style)` — arbitrary node-style overrides.
- `Op.StyleEdge(path, style)` — arbitrary edge-style overrides.
- `Op.Commit(alt)` — finalise the current frame, attach `alt` text to it, and open a fresh blank frame. `alt` is required so every frame the command stream produces carries accessible text.
- `Op.Alt(text)` — set alt text on the in-progress frame without committing. Primary use is the trailing frame, which has no following `Op.Commit` to carry its alt; can also be used to set alt earlier in a frame's lifecycle if convenient.
- `Op.ClearNotes()` — drop all notes from the current frame's snapshot.

Captions and step metadata are renderer-level rather than snapshot-level: set them between Op batches via `r.with-caption(...)` / `r.with-step(...)`, not through an Op. Fold a sequence of Ops into a renderer with `apply-ops()`.